DAST Scanner — PoC Técnico

Sep 24, 2026 · @Jader Paim

1. Visão geral e objetivo

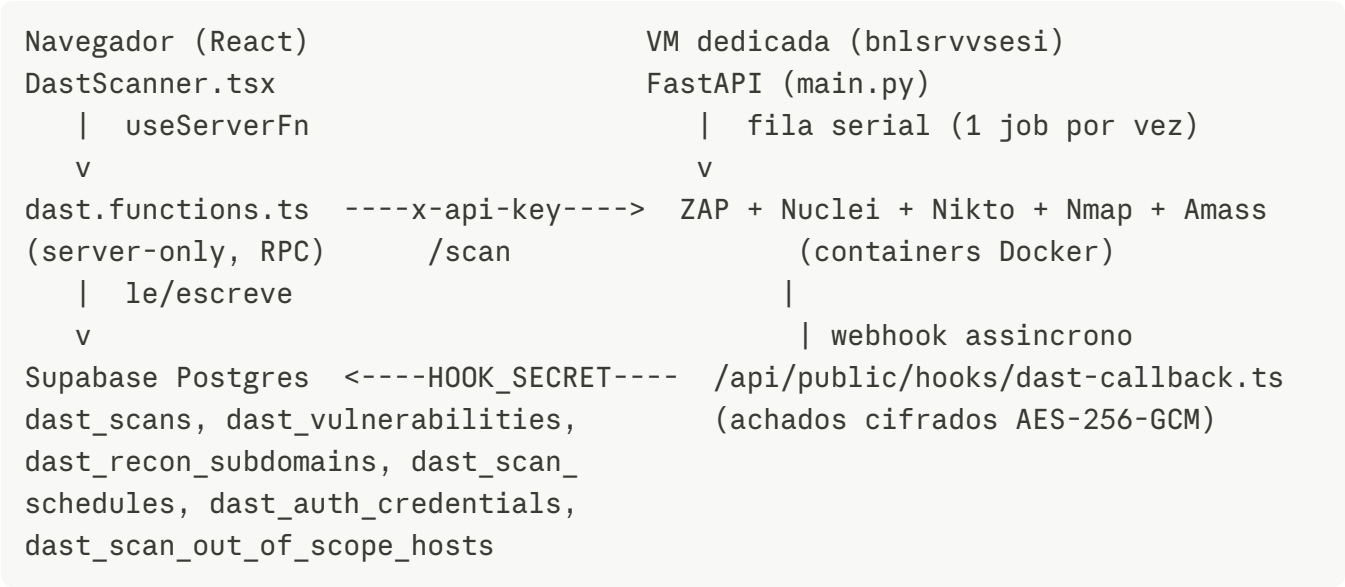
O **DAST Scanner** é o módulo de varredura dinâmica de segurança (Dynamic Application Security Testing) do b-security. Ele orquestra motores de scan de mercado — **OWASP ZAP**, **Nuclei**, **Nikto**, **Nmap** e **Amass** — contra aplicações e domínios da organização, encontra vulnerabilidades reais (não só lista CVEs teóricas), e apresenta tudo num painel unificado: acompanhamento ao vivo, histórico, retest, agendamento recorrente e relatório executivo em PDF.

Diferente de um scanner isolado rodando na máquina de um analista, o DAST Scanner é uma **plataforma contínua**: os motores rodam numa VM dedicada (Ubuntu, `bnlsrvvsesi`), orquestrados por um serviço FastAPI próprio, e os resultados voltam pro b-security via webhook autenticado — gerando um histórico persistente, deduplicado e auditável de cada vulnerabilidade encontrada, retestada, resolvida ou aceita como risco.

O objetivo declarado (refletido na própria UI) é cobrir o ciclo completo de um teste de segurança de aplicação: **descoberta de superfície** (quais domínios/subdomínios existem) → **varredura** (passiva e ativa, black box ou autenticada) → **triagem** (severidade, CVSS, OWASP, CWE) → **remediação e retest** → **relatório** — com uso interno restrito ao time de Segurança da Informação.

2. Arquitetura

O módulo tem três camadas, deliberadamente separadas:



Fluxo de um scan novo: o usuário preenche o diálogo “Novo scan” → `startDastScan` valida os alvos (guarda anti-SSRF, só aceita host/IP público ou faixa autorizada), resolve credencial white-box se pedido, aplica a troca de host “Projeto Lovable” se marcado → grava uma linha em `dast_scans` (`status: queued`) → chama o scanner via `/scan` (confirmação síncrona em segundos, o scan em si roda assíncrono) → o FastAPI processa a fila e, ao terminar (ou periodicamente durante scans longos), faz POST pro webhook com os achados — que decifra, deduplica via RPC (`upsert_dast_vulnerability`) e atualiza o status/progresso do scan. O front-end acompanha por polling (`refetchInterval`) e por Supabase Realtime nas tabelas `dast_scans` / `dast_vulnerabilities` .

Por que a VM é separada do app: os motores de scan (principalmente o ZAP, um daemon Java pesado) precisam rodar continuamente, consomem CPU/memória de forma imprevisível durante um ataque ativo, e não cabem no modelo serverless do resto do b-security. A VM também hospeda o módulo Shadow IT (mesma infra, dados totalmente separados — ver documento próprio) e, mais recentemente, o worker de automação SEK/Zendesk, todos coexistindo atrás do mesmo `main.py` .

Por que os achados chegam cifrados, não só em base64: achados de active scan carregam payload de ataque real (XSS/SQLi de teste). Mandar isso como JSON legível pro domínio protegido pela Cloudflare fazia o próprio WAF barrar a chamada de callback (confirmado em produção: 403 do Managed Ruleset, scan preso em “running” pra sempre). Base64 sozinho não bastava — o WAF decodifica transformações conhecidas antes de casar assinatura. A solução foi cifrar de ponta a ponta (AES-256-GCM) o payload de achados no callback.

3. Linha do tempo de construção

Data	Marco
10/07/2026	Módulo nasce: `DastScanner.tsx`, `dast.functions.ts`, `dast.server.ts`, `dast-callback.ts`, agendamento. Schema inicial (`dast_scans`, `dast_vulnerabilities`, `dast_recon_subdomains`, `dast_scan_schedules`, `dast_retest_history`) criado direto no banco, sem migration versionada.
21/08	Primeira onda de "engenharia de identidade": limpeza de duplicatas + coluna normalizada + `UNIQUE` + RPC `upsert_dast_vulnerability` — scans repetidos passam a **atualizar/reabrir** achados em vez de duplicar. Correção no mesmo dia pra templates Nuclei com múltiplas sub-extrações (`sub_key`).
24–25/08	Mesmo tratamento de dedup aplicado a `dast_recon_subdomains`; correção do normalizador de esquema (Amass grava com `https://`, Cloudflare grava sem — mesmo host virava 2 linhas); coluna `source` pra rastrear proveniência; `is_dast_allowed()` ganha suporte a `super_admin`.
28/08	**Separação de escopo**: descoberta de rede interna sai do DAST e vira o módulo **Shadow IT** (documento próprio). "Ativos Descobertos" do DAST passa a cobrir só descoberta externa.
10/09	Relatório PDF de verdade (`dast-report.ts`) — renderizado via Chromium headless na própria VM do scanner, substituindo o print-to-pdf do navegador (que quebrava layout de forma inconsistente).
11–14/09	**Scan autenticado (white box)** ganha vida: tabela `dast_auth_credentials` (credencial por domínio), campo `auth_status` no scan, URL de login opcional (com fallback pra raiz do alvo).

Data	Marco
21/09	Endurecimento de sessão em todo o app: `requireActiveSession` substitui o middleware anterior em todos os módulos, incluindo os ~25 server functions do DAST.
22–23/09	Tabela `dast_scan_out_of_scope_hosts` (domínios de terceiro vistos mas não atacados). Incidente de produção: scan pesado contra o próprio domínio estourou a cota do Worker Cloudflare (plano Free) — fix “Projeto Lovable” (troca de host no scan).
22–24/09	Sessão intensiva de hardening do motor de scan (detalhada na seção 5): rota de proxy do worker SEK perdida e restaurada, bug de formato do ZAP corrigido, contencão de recurso, `DomXssScanRule` identificada como causa de travamento e desabilitada, container do ZAP recriado com volume persistente, `--init` e heap maior.

Observação de proveniência: os commits deste repositório (gerado via Lovable) trazem mensagens automáticas de `git status`, não descritivas — a cronologia acima foi reconstruída a partir de datas de migration e comentários de código, não de mensagens de commit.

4. Capacidades atuais

Novo scan — URL avulsa ou lista de alvos; nível (básico/intermediário/avançado); **perfis prontos** (Rápido, Aplicação Web, Infraestrutura/Rede, Completo, Personalizado) que travam quais engines rodam; **black box ou white box** (autenticado, opt-in explícito — nunca automático só por existir credencial); opção “Projeto Lovable” pra bypassar o Worker Cloudflare do domínio próprio.

Descoberta de superfície — três caminhos independentes: **Externo** (Amass, OSINT público), **Cloudflare** (pull direto da zona DNS gerenciada — mais completo que Amass pra domínio com certificado wildcard, que não deixa rastro em Certificate Transparency), e **Interno via AD** (zonas DNS AD-integradas via LDAP, fluxo de 2 passos com confirmação manual de quais FQDNs viram ativo).

Acompanhamento ao vivo — scans ativos com barra de progresso e cancelamento real; alertas imediatos pra achados crítico/alto; filtro por severidade/status; **domínios de terceiro fora do escopo** (analytics/tracking/CDN vistos mas não atacados — puramente informativo).

Triagem de vulnerabilidade — cada achado traz severidade, CVSS, categoria OWASP, CWE (com link pra definição oficial), CVE (link pro NVD), evidência, payload/comando de reprodução, requisição/resposta bruta (Nuclei). Ações: **retest** (confirma se ainda é explorável), marcar resolvida, falso positivo, aceitar risco. Achados repetidos entre scans **reabrem** o mesmo registro (com histórico de ocorrências) em vez de duplicar.

Agendamento — scans recorrentes (diário/semanal/mensal, horário fixo), reaproveitando credencial de autenticação por domínio automaticamente.

Exportação — PDF (relatório executivo de verdade, renderizado via Chromium headless, com capa, resumo executivo, metodologia, achados detalhados e a seção de domínios fora de escopo quando aplicável), além de HTML simples, CSV e JSON.

Credenciais — gerenciamento próprio, cifrado (AES-256-GCM): credencial AD/LDAP (única, global, pra descoberta interna) e credenciais de autenticação **por domínio** (usuário/senha de teste, URL de login opcional, indicador de sessão logada) pra scan white box.

5. Motor de scan — engines e gotchas descobertos

O motor de scan roda 5 ferramentas de mercado, cada uma com um papel específico: **ZAP** (OWASP Zed Attack Proxy — crawler + scanner ativo/passivo, o mais completo e o mais complexo de operar), **Nuclei** (templates de CVE/misconfig conhecidos, rápido), **Nikto** (varredura de servidor web), **Nmap** (portas/serviços) e **Amass** (enumeração de subdomínio via OSINT).

O ZAP concentra a maior parte da complexidade operacional porque é um **daemon único e compartilhado** (não sobe um container novo por scan, como os outros) — isso trouxe uma série de comportamentos reais, não documentados, que precisaram ser descobertos em produção:

- `scanAsUser` **devolve o ID do scan numa chave diferente** do modo anônimo — um código que só checava a chave padrão falhava silenciosamente em todo scan autenticado (achava que nunca começou, pulava o acompanhamento, o scan continuava rodando sozinho por horas sem ninguém saber).

- **O Ajax Spider pede o NOME do usuário/contexto, não o ID** — única das três operações (spider/active scan/ajax spider) com essa diferença.
- **Autenticação via navegador real + Spider multi-thread corrompe o daemon** — cada login sobe um servidor local pro callback; threads concorrentes autenticando ao mesmo tempo colidem e derrubam o processo do ZAP inteiro. Corrigido limitando a 1 thread durante scan autenticado.
- **Add-on "Insights" desligava o daemon inteiro sozinho** — um mecanismo de auto-proteção que, ao detectar padrão de falha repetida, mata o processo ZAP inteiro (não só o scan problemático). Removido de propósito — já existe proteção equivalente e mais cirúrgica no código próprio.
- **Cancelamento de scan não estava matando o container de verdade** — `proc.kill()` sozinho só mata o cliente CLI local do `docker run`, não o container por trás dele (confirmado ao vivo: container Nuclei continuou rodando 5+ minutos depois de "cancelado"). Corrigido dando nome explícito a cada container e chamando `docker kill` de verdade, tanto no cancelamento quanto em todo timeout interno de engine.
- **Contencão de recurso entre alvos** — o passive scan do ZAP roda numa thread pool própria, independente da fila do app; o backlog de mídia/tracking de terceiro de um alvo dispute CPU com o active scan do próximo. Corrigido com exclusão global de mídia estática e trackers conhecidos (tanto no Spider clássico quanto no proxy, que é o que o Ajax Spider realmente respeita) e uma espera limitada (5min) pro passive scan esvaziar antes de avançar.
- **`DomXssScanRule` travava o active scan inteiro** — essa regra abre um navegador real pra instrumentar DOM XSS; quando a configuração da extensão do ZAP no navegador falhava (confirmado: falha sistemática nesse ambiente, mesmo com Chromium instalado corretamente), a thread principal do scan travava sem devolver controle — dois alvos diferentes do mesmo job travaram no **mesmo 24%** de progresso, cada um consumindo 45–70 minutos até o timeout de segurança (20min de inatividade) desistir. Desabilitada globalmente; outras técnicas de XSS (refletido/persistente) continuam cobrindo normalmente.
- **814 processos zumbi acumulados** — causa raiz real: o container do ZAP rodava sem `--init`; processos de navegador órfãos (das falhas acima) nunca eram colhidos porque o processo Java é o PID 1 do container e não sabe fazer isso sozinho. Resolvido recriando o container com `--init`.
- **Vídeos e arquivos grandes (>16MB) quebravam o histórico interno do ZAP** — limite de tamanho de resposta configurável só na inicialização do daemon, não em runtime; elevado pra 100MB na recriação do container.

- **Teto de cache do HSQLDB (32000 linhas), sem chave de configuração exposta.** Confirmado ao vivo (24/09): sessão única acumulando histórico de vários alvos pesados estoura um limite interno do banco embutido do ZAP, travando toda gravação nova do Spider dali em diante — a causa concreta por trás da degradação progressiva em scans bulk longos. Corrigido chamando `core/action/newSession` antes de cada alvo (zera o histórico; achados já extraídos pro Supabase antes disso, nada se perde) — as exclusões de mídia/tracking são escopo de sessão e se perdem com o reset (confirmado ao vivo), então são reaplicadas logo em seguida; o `DomXssScanRule` desabilitado é política de scan, não sessão, e sobrevive sozinho.

O container do ZAP hoje roda com **volume persistente** (add-ons e configuração sobrevivem a restart), **8GB de heap**, `--init` e sessão nova a cada alvo (`core/action/newSession`, evita estourar o teto de cache do HSQLDB) — as quatro mudanças mais recentes, todas aplicadas em 24/09.

6. Incidentes reais e correções (fora do motor de scan)

Cota do Worker Cloudflare derrubou produção. Um scan pesado apontado contra o próprio domínio (`b-security.bernoulli.com.br`, atrás de um Worker no plano Free, cota de 100mil requisições/dia) estourou a cota — causou Erro 1027 (rate limit), depois uma tentativa de contornar via DNS sem proxy resultou em Erro 1034 (Edge IP Restricted), e por fim 429 genocórico em **todo** o tráfego do site, incluindo os próprios callbacks do scanner e as ações do usuário no navegador. Corrigido revertendo o DNS e construindo o recurso "Projeto Lovable": scans marcados com essa opção trocam o host real (`*.bernoulli.com.br`) pelo domínio espelho na Lovable (`*.lovable.app`) só pro tráfego do scan, sem passar pelo Worker.

WAF bloqueando o próprio callback de resultado. Achados de active scan carregam payload de ataque real; a Cloudflare identificava esse conteúdo como tentativa de XSS/injeção de verdade e bloqueava a chamada de callback com 403, deixando o scan preso em "running" indefinidamente. Corrigido cifrando o payload de achados de ponta a ponta (AES-256-GCM) em vez de só base64 — o WAF não decifra, então não casa assinatura de ataque contra um blob cifrado.

Rota de proxy compartilhada perdida num deploy. O `main.py` do scanner também hospeda um proxy reverso pro worker de automação SEK/Zendesk (infraestrutura separada, ver documentação própria daquele projeto), que roda na mesma VM. Um deploy do `main.py` durante os ajustes do DAST sobrescreveu o arquivo inteiro e apagou essa rota sem querer (deploy nessa VM é sempre substituição completa do arquivo, não

um patch) — detectado por um 404 inesperado em produção, restaurado a partir do código original. Fica como lição operacional: qualquer sessão futura que edite esse arquivo precisa comparar contra a versão rodando na VM antes de reenviar.

Relatório de domínios de terceiro fora do escopo. Decisão de produto de 22/09: em vez de só excluir silenciosamente analytics/tracking/CDN de terceiro do rastreamento, o relatório passou a listar quais domínios de terceiro foram encontrados mas não atacados — dá base de auditoria e permite "curar" a lista de exclusão ao longo do tempo (identificar um domínio que aparece repetidamente e ainda não está excluído).

7. Segurança e controle de acesso

Quem pode acessar: a função `is_dast_allowed()` exige `admin` ou `super_admin` **e** que o usuário pertença ao departamento "Segurança da Informação" (comparação insensível a acento). É checada em todo server function (backend, boundary real) e espelhada só por UX na tela de bloqueio do front-end. O módulo Shadow IT reaproveita essa mesma função em vez de ter grupo de acesso próprio.

Comunicação app ↔ scanner: toda chamada do b-security pro FastAPI leva um `x-api-key` (guardado cifrado na tabela `integrations`), mais cabeçalhos opcionais de Cloudflare Access quando aplicável. O caminho inverso (scanner → app, via webhook) é protegido por `HOOK_SECRET` compartilhado, checado antes de qualquer parsing do corpo.

Credenciais nunca em texto claro: credencial AD/LDAP, credenciais de autenticação por domínio e a chave da API do scanner são cifradas com AES-256-GCM antes de gravar no banco; os endpoints de "status" devolvem só usuário/configúido, nunca a senha. Credencial de scan autenticado trafega pro scanner por requisição (decifrada só no momento do uso), nunca fica persistida na VM.

Guarda anti-SSRF: alvos de scan passam por validação antes de aceitar o job — evita que o próprio scanner seja usado como vetor pra atacar infraestrutura interna não autorizada.

Auditoria: toda ação que muda estado (criar/cancelar/excluir scan, salvar credencial, mudar status de achado) grava em log de auditoria, com o módulo identificado.

Lacuna de governança documentada, não escondida: a função `is_dast_allowed()` foi criada direto no banco em algum momento antes do controle de versão começar a rastrear esse módulo — não existe migration de criação dela no repositório. A primeira alteração registrada (25/08, adicionar `super_admin`) documenta explicitamente que a definição foi capturada do banco antes de alterar, pra preservar o comportamento e só adicionar o necessário.

8. Limitações conhecidas e pendências

- **Importadores de spec (OpenAPI/Postman/GraphQL/SOAP) instalados mas não usados.** Os quatro add-ons do ZAP estão presentes e prontos, mas o `main.py` nunca os chama — falta um campo novo na UI ("URL da spec, opcional") e o encanamento até o scanner. Escopo claro, só não priorizado ainda.
- **DOM XSS via instrumentação de navegador desabilitado.** Perdemos essa técnica específica de detecção (regra `DomXssScanRule`) por instabilidade real do ambiente — XSS refletido/persistente continuam cobertos por outras regras, mas DOM XSS via análise dinâmica de navegador, não.
- **Degradação em sessão única muito longa — causa confirmada, fix aplicado (24/09).** Causa raiz confirmada ao vivo: o HSQLDB do ZAP tem um teto de cache fixo (32000 linhas, sem chave de configuração exposta) — sessão única acumulando histórico de vários alvos pesados estoura esse teto, travando gravação nova no Spider. Corrigido chamando core/action/newSession antes de cada alvo (achados já extraídos pro Supabase antes disso, nada se perde) e reaplicando as exclusões de mídia/tracking logo depois (escopo de sessão, se perdem com o reset, confirmado ao vivo). Heap também triplicado como reforço; ainda não validado num scan real de ponta a ponta. Recomendação prática enquanto isso: preferir lotes menores (3–4 alvos) a um scan único com 9+.
- **`is_dast_allowed()` sem migration de criação no repositório** — lacuna de rastreabilidade histórica, já documentada, sem risco funcional imediato.
- **Sem grupo de permissão próprio** — acesso depende 100% de cargo + departamento; qualquer mudança organizacional nesses dois campos afeta o acesso ao módulo diretamente.
- **Callback não deduplica domínios fora do escopo entre scans repetidos do mesmo alvo** — cada execução grava sua própria leitura; consultas de "esse domínio aparece toda vez" precisam agregar na consulta, não vêm prontas.
- **Segredo do agendamento (`HOOK_SECRET`) gravado em claro no comando de cron do Postgres** — pendência de segurança conhecida, fix planejado via Supabase Vault, ainda não implementado (compartilhada com outros módulos que usam o mesmo padrão de agendamento).

9. Estado atual e conclusão

O DAST Scanner completou, em pouco mais de dois meses e meio (10/07 a 24/09), o percurso de um recurso recém-lançado até uma plataforma de varredura contínua razoavelmente madura: descoberta de superfície em três modos, scan black box e white box, dedup/reabertura de achados, agendamento, relatório executivo em PDF e uma camada real de resiliência operacional (cancelamento que mata processo de verdade, contencão de recurso entre alvos, recuperação de travamentos conhecidos do ZAP).

A sessão mais recente (22–24/09) fechou um ciclo importante de estabilização de infraestrutura — boa parte dos problemas encontrados não eram bugs do código do b-security, e sim comportamentos reais e mal documentados do próprio ZAP e do ambiente Docker em que ele roda, só descobíveis testando contra tráfego real. Com o container do ZAP recriado (volume persistente, `--init`, heap maior) e sessão nova por alvo (evita o teto de cache do HSQLDB que causava boa parte dos travamentos em % avançado), a causa raiz dos travamentos foi identificada e corrigida no mesmo dia — inclusive um pequeno incidente no processo, registrado como lição: um teste manual isolado acabou derrubando a sessão de um scan que já estava rodando. O próximo scan bulk grande é o teste real de ponta a ponta dessas mudanças.

Pendências em aberto são conhecidas, documentadas e priorizadas (seção 8) — nenhuma bloqueia o uso atual do módulo pelo time de Segurança da Informação.